```
class DriversLicenseValidator < ActiveModel::EachValidator
  def validate_each(record, attribute, value)
  end
end
```

A completed Validator for validating the format of Drivers License Numbers has been added to the Rideshare codebase in the app/validators directory:

ruby/drivers_license_number_validator.rb

```
class DriversLicenseValidator < ActiveModel::EachValidator

  DL_MN_REGEXP_FORMAT = /[a-zA-Z]\d{12}/i
  DEFAULT_MESSAGE = "is not a valid driver's license number"

  def validate_each(record, attribute, value)
    unless value =~ DL_MN_REGEXP_FORMAT
      record.errors.add(
        attribute,
        options[:message] || DEFAULT_MESSAGE
      )
    end
  end
end
```

Drivers are required to have a Driver's License and their license number should be unique. Use the built-in presence and uniqueness validations in addition to the Custom Driver's License Number Validator for the drivers_license_number field.

The Driver model in Rideshare combines these validators:

ruby/driver_model.rb

```
class Driver < User
  validates :drivers_license_number,
    presence: true,
    uniqueness: true,
    drivers_license: true
end
```

You've now seen how multiple Active Record validations can be used together. Pretty slick. You've made sure the data is populated, unique, and matches an expected format.

Although these rules are rigid; at other times you may want flexibility in how you accept input from the user.

One example is case sensitivity vs. insensitivity. Email addresses are not case sensitive. However, email addresses may be entered as user input using specific casing and you may wish to preserve that casing for presentation